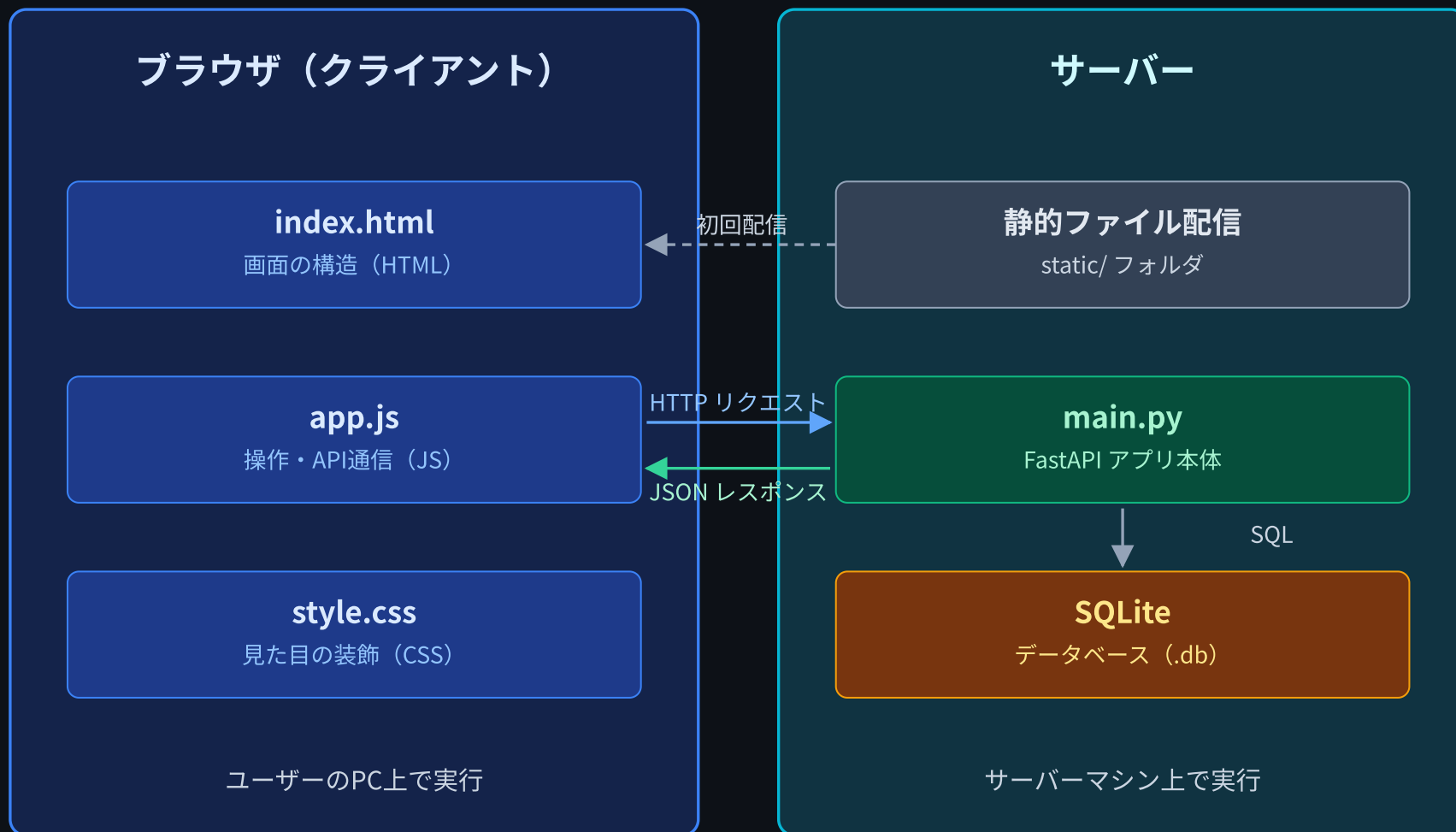


第6回: データベース (SQLite) & REST API

Webアプリケーション基礎 2026

今日のゴール

サーバーにCRUD操作のAPIを完成させる



今日の流れ

前半

- データベースとは / SQLiteの特徴
- SQL基礎 — CREATE TABLE, INSERT INTO
- SQL基礎 — SELECT, UPDATE, DELETE

後半

- REST APIの設計
- FastAPI + SQLite連携 — GET / POST
- FastAPI + SQLite連携 — PUT / DELETE / Swagger UI

1. データベースとは

なぜデータベースが必要なのか？

変数に保存する場合の問題

```
todos = [  
  {"id": 1, "title": "買い物", "done": False},  
  {"id": 2, "title": "宿題", "done": True},  
]
```

- サーバーを再起動すると データが消える
- プログラムが終了すると メモリが解放 される
- 複数のユーザーが同時にアクセスすると データの整合性 が取れない

ファイル保存ではダメなのか？

JSONファイルに保存する場合の問題

```
import json
with open("todos.json", "w") as f:
    json.dump(todos, f)
```

- 毎回 **ファイル全体** を読み書きする必要がある（大量データで遅い）
- 「IDが3のTODOだけ取得」のような **部分検索が非効率**
- 複数ユーザーが同時に書き込むと **データが壊れる** 可能性
- データの **構造（型）を強制** できない

データベースが解決すること

問題	データベースの解決策
サーバー再起動でデータ消失	ディスクに永続的に保存
大量データの検索が遅い	インデックスによる高速検索
同時アクセスでの不整合	トランザクション管理
データ構造の強制なし	スキーマ（テーブル定義）で型を強制
部分的な取得・更新が困難	SQLで柔軟にクエリ

データベースは「データを安全・高速・効率的に管理する専用ソフトウェア」です。

SQLiteとは

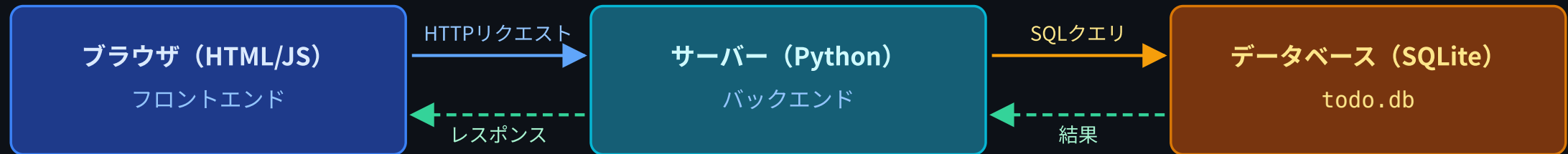
特徴

- ファイル1つ で動くデータベース（ `todo.db` というファイルがDB本体）
- サーバー不要（インストール不要で手軽）
- Pythonに 標準搭載（ `import sqlite3` で使える）
- 小～中規模のアプリケーションに最適

重要なポイント

- データベースは サーバー側（バックエンド）でのみ動く
- ブラウザ（フロントエンド）から直接アクセスすることはない
- API経由でデータをやり取りする

データベースの位置づけ



ブラウザ → サーバー → データベース の順番でデータが流れます。

実習1: SQLiteを起動してみよう

手順

1. ターミナルで以下を実行:

```
cd session06/exercise  
sqlite3 todo.db
```

2. SQLiteのプロンプト `sqlite>` が表示されたら成功

3. テーブル一覧を確認:

```
.tables
```

→ まだ何もないので空のまま（これからテーブルを作ります）

4. SQLiteの終了:

```
.quit
```

2. SQL基礎 - CREATE TABLE / INSERT INTO

テーブルとは

テーブル = Excelのシート

id	title	done
1	買い物	0
2	宿題	1
3	掃除	0

- テーブル = データを格納する表 (Excelのシートに相当)
- カラム (列) = データの種類 (id, title, done)
- レコード (行) = 1件のデータ
- データベース = テーブルの集まり (Excelのブックに相当)

CREATE TABLE - テーブルを作る

```
CREATE TABLE todos (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  title TEXT NOT NULL,  
  done INTEGER DEFAULT 0  
);
```

各部分の解説

要素	意味
CREATE TABLE todos	todosという名前のテーブルを作成
id INTEGER PRIMARY KEY AUTOINCREMENT	自動で1, 2, 3...と増える番号
title TEXT NOT NULL	テキスト型、空を許さない
done INTEGER DEFAULT 0	整数型、初期値は0（未完了）

- PRIMARY KEY = 各レコードを一意に識別するキー
- AUTOINCREMENT = 自動で番号を振ってくれる

SQLiteのデータ型

データ型	説明	例
INTEGER	整数	1, 42, 0
TEXT	文字列	'こんにちは', 'hello'
REAL	小数	3.14, 0.5
BLOB	バイナリデータ	画像データなど

注意点

- SQLiteでは 真偽値 (boolean) は INTEGER で表現する
 - 0 = False (未完了)
 - 1 = True (完了)

INSERT INTO - データを追加する

```
-- 1件追加 (titleだけ指定、idは自動、doneは初期値0)
INSERT INTO todos (title) VALUES ('買い物に行く');

-- 複数のカラムを指定して追加
INSERT INTO todos (title, done) VALUES ('宿題をする', 0);

-- doneを1 (完了) にして追加
INSERT INTO todos (title, done) VALUES ('部屋の掃除', 1);
```

ポイント

- `id` は `AUTOINCREMENT` なので 書かなくてOK (自動で振られる)
- 文字列は シングルクォート `'...'` で囲む
- `--` はSQLのコメント

実習2: テーブルを作成してデータを追加しよう

手順

1. SQLiteを起動:

```
sqlite3 todo.db
```

2. todosテーブルを作成:

```
CREATE TABLE todos (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    title TEXT NOT NULL,  
    done INTEGER DEFAULT 0  
);
```

実習2: テーブルを作成してデータを追加しよう (続き)

3. テーブルができたか確認:

```
.tables
```

4. 3件のデータを追加:

```
INSERT INTO todos (title) VALUES ('買い物に行く');  
INSERT INTO todos (title) VALUES ('レポートを書く');  
INSERT INTO todos (title, done) VALUES ('部屋の掃除', 1);
```

3. SQL基礎 - SELECT / UPDATE / DELETE

SELECT - データを取得する

```
-- 全件取得
SELECT * FROM todos;

-- 特定のカラムだけ取得
SELECT id, title FROM todos;

-- 条件を指定して取得 (WHERE句)
SELECT * FROM todos WHERE done = 0;

-- 完了済みのTODOだけ取得
SELECT * FROM todos WHERE done = 1;

-- IDを指定して1件取得
SELECT * FROM todos WHERE id = 2;
```

ポイント

- `*` は「全てのカラム」を意味する
- `WHERE` で条件を絞り込む
- `=` は「等しい」(Pythonの `==` に相当)

UPDATE - データを更新する

```
-- IDが1のTODOを完了にする
UPDATE todos SET done = 1 WHERE id = 1;

-- IDが2のTODOのタイトルを変更
UPDATE todos SET title = '数学のレポートを書く' WHERE id = 2;

-- 全てのTODOを未完了にする (WHERE なし = 全件対象)
UPDATE todos SET done = 0;
```

注意！

- **WHERE** を忘れると全てのレコードが更新される！
- 必ず **WHERE** で対象を絞り込むこと

DELETE - データを削除する

```
-- IDが3のTODOを削除
DELETE FROM todos WHERE id = 3;

-- 完了済みのTODOを全て削除
DELETE FROM todos WHERE done = 1;

-- 全件削除 (WHERE なし = 全件対象)
DELETE FROM todos;
```

注意！

- **WHERE** を忘れると全てのレコードが削除される！
- 削除したデータは 元に戻せない

CRUD操作のまとめ

操作	SQL文	意味
Create（作成）	INSERT INTO	新しいデータを追加
Read（読み取り）	SELECT	データを取得
Update（更新）	UPDATE	既存データを変更
Delete（削除）	DELETE	データを削除

CRUD はデータ操作の基本4操作です。

ほとんどのアプリケーションは、この4つの組み合わせで成り立っています。

実習3: データの取得・更新・削除を試そう

手順

1. 全件取得:

```
SELECT * FROM todos;
```

2. 未完了のTODOだけ取得:

```
SELECT * FROM todos WHERE done = 0;
```

3. IDが1のTODOを完了にする:

```
UPDATE todos SET done = 1 WHERE id = 1;
```


実習3: データの取得・更新・削除を試そう (続き)

4. 更新されたか確認:

```
SELECT * FROM todos;
```

5. IDが1のTODOを削除:

```
DELETE FROM todos WHERE id = 1;
```

6. 削除されたか確認して、`.quit` で終了

4. REST APIの設計

REST APIとは

RESTの原則

- REpresentational State Transfer の略
- URLで「何（リソース）」を指定し、HTTPメソッドで「どうする（操作）」を指定

HTTPメソッド

メソッド	意味	CRUD対応
GET	取得する	Read
POST	作成する	Create
PUT	更新する	Update
DELETE	削除する	Delete

URLとメソッドの組み合わせで、何をすることが決まります。

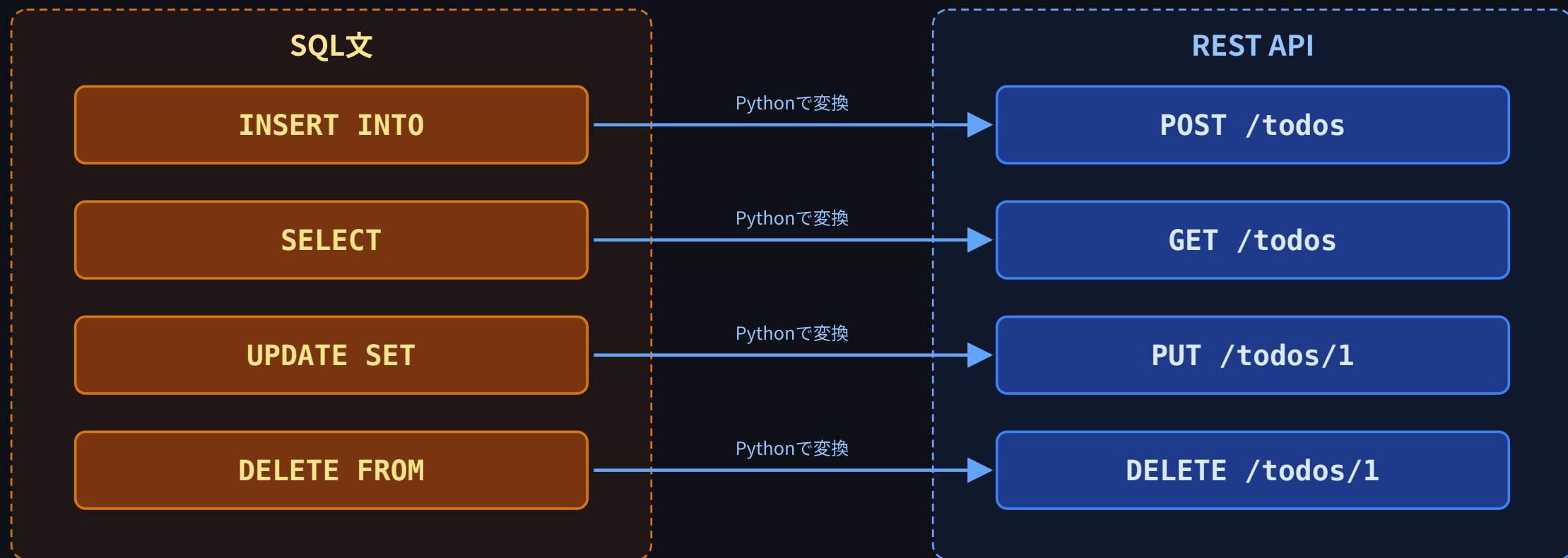
TODO APIの設計

操作	HTTPメソッド	エンドポイント	説明
一覧取得	GET	/todos	全TODOを取得
新規追加	POST	/todos	新しいTODOを追加
完了切替	PUT	/todos/{id}	指定IDのTODOのdoneを更新
削除	DELETE	/todos/{id}	指定IDのTODOを削除

ポイント

- {id} は パスパラメータ（実際には /todos/1 , /todos/2 のようになる）
- 一覧操作は /todos、個別操作は /todos/{id} とURLを統一

CRUDとHTTPメソッドの対応図



SQL操作とHTTPメソッドが **1対1**で対応しています。

APIのリクエストとレスポンスの具体例

GET /todos (一覧取得)

```
// レスポンス
[
  {"id": 1, "title": "買い物", "done": false},
  {"id": 2, "title": "宿題", "done": true}
]
```

POST /todos (新規追加)

```
// リクエストボディ
{"title": "新しいTODO"}

// レスポンス
{"id": 3, "title": "新しいTODO", "done": false}
```

実習4: API設計書を作成しよう

手順

1. `session06/exercise/api_design.md` を開く
2. テンプレートの空欄を埋めて、TODO APIの設計書を完成させる
3. 各APIについて以下を記述:
 - HTTPメソッド
 - エンドポイント (URL)
 - リクエストの形式
 - レスポンスの形式

この設計書を基に、次のセグメントでコードを実装します。

5. FastAPI + SQLite連携 - GET / POST

PythonからSQLiteを使う

sqlite3モジュール（Python標準ライブラリ）

```
import sqlite3

# データベースに接続
conn = sqlite3.connect("todo.db")
# カーソルを取得（SQLを実行するオブジェクト）
cursor = conn.cursor()

# SQLを実行
cursor.execute("SELECT * FROM todos")
rows = cursor.fetchall() # 全件取得

# 接続を閉じる
conn.close()
```

ポイント

- `connect()` でDBファイルに接続（ファイルがなければ自動作成）
- `cursor()` でSQL実行用のオブジェクトを取得
- `execute()` でSQLを実行
- `fetchall()` で結果を全件取得

Pydantic BaseModel - リクエストの型定義

```
from pydantic import BaseModel

class TodoCreate(BaseModel):
    title: str

class TodoUpdate(BaseModel):
    done: bool
```

Pydanticとは？

- リクエストの データ形式を定義 するライブラリ
- FastAPIが自動で バリデーション（型チェック） してくれる
- 不正なデータが来たら自動でエラーを返す

例

- `TodoCreate` → POSTリクエストで `{"title": "買い物"}` を受け取る
- `TodoUpdate` → PUTリクエストで `{"done": true}` を受け取る

DBへの接続は各APIの中で行う

```
import sqlite3

DATABASE = "todo.db"  # データベースファイルの名前
```

- 各APIの中で `sqlite3.connect(DATABASE)` して接続し、処理後に `conn.close()` で閉じる
- ヘルパー関数や特別な設定は使わず、「毎回つなぐ」シンプルな形に統一する

取得した結果はタプルで返る

```
# fetchall() / fetchone() で取り出した1行はタプル
# (1, '買い物', 0)

# 番号（インデックス）で取り出す
# row[0] → id, row[1] → title, row[2] → done
```

番号の取り違えを防ぐため、`SELECT *` ではなく

`SELECT id, title, done` のように **必要なカラムを順番どおりに指定** する。

GET /todos の実装

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/todos")
def list_todos():
    conn = sqlite3.connect(DATABASE)
    cursor = conn.cursor()
    cursor.execute("SELECT id, title, done FROM todos ORDER BY id")
    rows = cursor.fetchall()
    conn.close()

    # 1行は (id, title, done) の順のタプル。番号で取り出して辞書のリストにする
    return [
        {"id": row[0], "title": row[1], "done": bool(row[2])}
        for row in rows
    ]
```

- `@app.get("/todos")` → GET /todos のエンドポイント
- SQLで全件取得し、JSON形式のリストとして返す

POST /todos の実装

```
from fastapi import FastAPI, HTTPException

@app.post("/todos")
def create_todo(todo: TodoCreate):
    conn = sqlite3.connect(DATABASE)
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO todos (title, done) VALUES (?, 0)",
        (todo.title,)
    )
    conn.commit()
    new_id = cursor.lastrowid
    conn.close()
    return {"id": new_id, "title": todo.title, "done": False}
```

ポイント

- `?` は プレースホルダ (SQLインジェクション対策)
- `conn.commit()` で変更を確定 (INSERT/UPDATE/DELETEの後に必要)
- `cursor.lastrowid` で自動採番されたIDを取得

SQLインジェクション対策

危険な書き方（絶対にやらない！）

```
# ユーザー入力をそのまま埋め込む → 危険！  
cursor.execute(f"INSERT INTO todos (title, done) VALUES ('{todo.title}', 0)")
```

安全な書き方（プレースホルダを使う）

```
# ? にパラメータを渡す → 安全！  
cursor.execute("INSERT INTO todos (title, done) VALUES (?, 0)", (todo.title,))
```

- ユーザーが `' ; DROP TABLE todos; --` のような入力をした場合
 - 危険な書き方 → テーブルが削除される！
 - 安全な書き方 → ただの文字列として処理される

実習5: GET/POSTを実装しよう

手順

1. まずDBを初期化:

```
cd session06/exercise  
python init_db.py
```

2. `exercise/main.py` を開く (GET /todos は実装済み)

3. POST /todos の `# TODO` コメント部分を実装

4. サーバーを起動して動作確認:

```
python main.py
```

5. ブラウザで `http://localhost:8000/todos` を開いてデータを確認

6. FastAPI + SQLite連携 - PUT / DELETE

PUT /todos/{id} の実装

```
@app.put("/todos/{todo_id}")
def update_todo(todo_id: int, todo: TodoUpdate):
    conn = sqlite3.connect(DATABASE)
    cursor = conn.cursor()

    # まず対象のTODOが存在するか確認
    cursor.execute("SELECT title FROM todos WHERE id = ?", (todo_id,))
    existing = cursor.fetchone()
    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="TODO not found")

    # doneを更新
    cursor.execute(
        "UPDATE todos SET done = ? WHERE id = ?",
        (int(todo.done), todo_id)
    )
    conn.commit()
    conn.close()

    # existing は (title,) のタプルなので先頭を取り出す
    return {
        "id": todo_id,
        "title": existing[0],
        "done": todo.done
    }
```

DELETE /todos/{id} の実装

```
@app.delete("/todos/{todo_id}")
def delete_todo(todo_id: int):
    conn = sqlite3.connect(DATABASE)
    cursor = conn.cursor()

    # まず対象のTODOが存在するか確認
    cursor.execute("SELECT id FROM todos WHERE id = ?", (todo_id,))
    existing = cursor.fetchone()
    if existing is None:
        conn.close()
        raise HTTPException(status_code=404, detail="TODO not found")

    # 削除
    cursor.execute("DELETE FROM todos WHERE id = ?", (todo_id,))
    conn.commit()
    conn.close()

    return {"message": "TODO deleted", "id": todo_id}
```

HTTPException - エラーハンドリング

```
from fastapi import HTTPException

# 存在しないIDが指定された場合
raise HTTPException(status_code=404, detail="TODO not found")
```

よく使うステータスコード

コード	意味	使用場面
200	OK	正常（GETの成功）
201	Created	作成成功（POSTの成功）
404	Not Found	リソースが見つからない
422	Unprocessable Entity	リクエストの形式が不正
500	Internal Server Error	サーバー内部エラー

Swagger UI - APIドキュメント

FastAPIが自動で生成するAPIドキュメント

- サーバー起動後、ブラウザで `/docs` にアクセス

```
http://localhost:8000/docs
```

Swagger UIでできること

- 全APIエンドポイントの一覧を確認
- 各APIのリクエスト/レスポンスの形式を確認
- 「Try it out」 ボタンで実際にAPIを実行できる
- curlコマンドやHTTPIe等を使わなくてもテスト可能

Swagger UIの使い方

1. APIを選択

- 実行したいAPIをクリックして展開

2. 「Try it out」 をクリック

- パラメータ入力欄が編集可能になる

3. パラメータを入力

- POST/PUTの場合はRequest bodyにJSONを入力

```
{"title": "Swagger UIから追加したTODO"}
```

4. 「Execute」 をクリック

- APIが実行され、レスポンスが表示される

5. レスポンスを確認

- ステータスコード、レスポンスボディを確認

全体のコード構成

```
session06/  
├── exercise/  
│   ├── init_db.py      ← DBの初期化（テーブル作成＋サンプルデータ）  
│   ├── main.py         ← FastAPIアプリ（実習で完成させる）  
│   ├── api_design.md   ← API設計書テンプレート  
│   ├── requirements.txt ← 依存パッケージ  
│   └── todo.db         ← SQLiteデータベース（init_db.pyで作成）
```


実習6: PUT/DELETEを実装しSwagger UIで確認

手順

1. `exercise/main.py` の PUT と DELETE の `# TODO` コメント部分を実装
2. サーバーを再起動(`ctrl+c` → `python main.py`)
3. Swagger UIで動作確認: `http://localhost:8000/docs`
4. 以下を試す:
 - GET /todos → 一覧取得
 - POST /todos → 新規追加
 - PUT /todos/1 → 完了切替
 - DELETE /todos/1 → 削除

実習6: Git commit & push

5. Git commit & push:

```
git add -A  
git commit -m "feat: 第6回 REST API完成"  
git push
```

本日のまとめ

学んだこと

- データベース の必要性和SQLiteの特徴
- SQL の基本操作 (CREATE, INSERT, SELECT, UPDATE, DELETE)
- CRUD操作 とHTTPメソッドの対応
- REST API の設計方法
- FastAPI + SQLite でのCRUD API実装
- Swagger UI でのAPI動作確認

次回予告

- 第7回: フロントエンドとバックエンドの結合
 - fetch APIでサーバーと通信
 - TODOアプリの完成！

提出物

実習で穴埋め・実装を完了させたファイルをフォームから提出してください:

1. `api_design.md` のGitHubのURL (実習4で完成させた設計書)
 - 例: `https://github.com/ユーザー名/リポジトリ名/blob/main/session06/exercise/api_design.md`
2. `main.py` のGitHubのURL (実習5・6完了後の状態 = GET / POST / PUT / DELETE のCRUD全API実装済み)
 - 例: `https://github.com/ユーザー名/リポジトリ名/blob/main/session06/exercise/main.py`